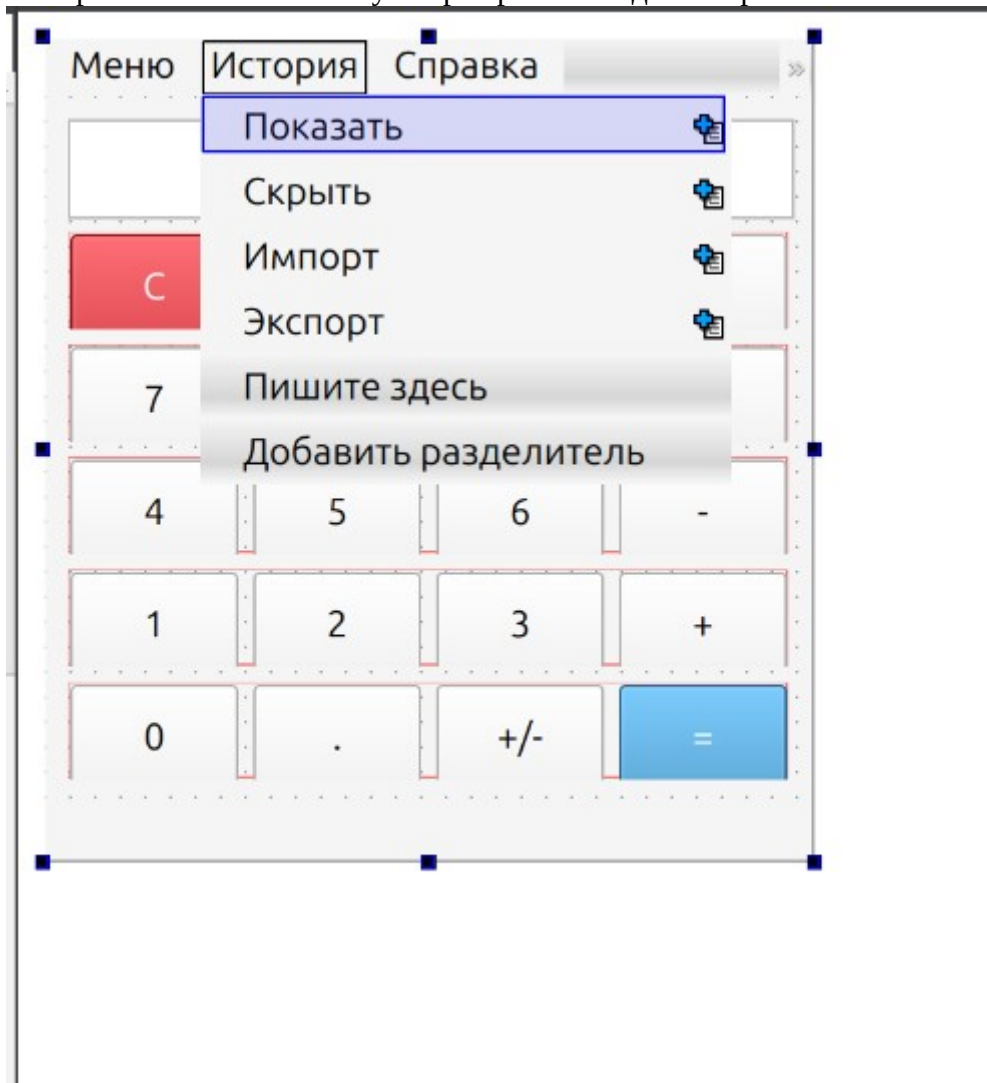


Практическая работа №1

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке
<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Добавлена строка состояния и статус-бар через меню дизайнера:



2) Добавлены слоты для обработки пунктов меню:

```

21
22 public:
23     explicit MainWindow(QWidget *parent = 0);
24     ~MainWindow();
25
26 private slots:
27     void onSaveAction();
28     void onShowHistory();
29     void onHideHistory();
30     void onClearHistory();
31     void onAbout();
32     void onHistoryItemClicked (QListWidgetItem *item);
33     void clearStatusBarMessage();
34     void updateHistoryList();
35     //Доп Задание
36     void onExport ();
37     void onImport ();
38

```

3) Объявлены переменные для элементов интерфейса виджета истории.

```

63
64     QList<QString> historyList;
65
66     QWidget *historyWidget;
67     QListWidget *historyView;
68     QPushButton *hideHistoryButton;
69     QPushButton *clearHistoryButton;
70     QVBoxLayout *historyLayout;
71     // QTimer *statusBarTimer;
72

```

QTimer был убран из функционала по причине наличия у StatusBar собственного таймера, выполняющего те же функции:

```

148     }
149     historyList.append(currentValue);
150     ui->statusBar->showMessage("Значение " + currentValue + " добавлено в историю.", 3000);
151

```

4) Добавлен виджет historyWidget, который создается динамически в коде программы, ему заданы размеры 200x400, в качестве основного компоновщика выбран вертикальный компоновщик, в него добавлены две кнопки «Очистить» и «Скрыть», а также виджет списка для отображения списка сохраненных вычислений. Для отображения нового виджета рядом с основным окном калькулятора применяется сплиттер. В нем устанавливается ограничение на растяжение для второго виджета — истории — и запрет на уменьшение размера виджета меньше чем минимальный размер. Созданный в итоге сплиттер устанавливается в качестве основного компоновщика окна, а виджет истории скрывается.

```

31
32 historyWidget = new QWidget(this);
33 historyWidget->setWindowTitle("История");
34 historyWidget->resize(200,400);
35
36 historyView = new QListWidget(historyWidget);
37 hideHistoryButton = new QPushButton("Скрыть", historyWidget);
38 clearHistoryButton = new QPushButton("Очистить", historyWidget);
39
40 QVBoxLayout *mainLayout = new QVBoxLayout(historyWidget);
41 mainLayout->addWidget(clearHistoryButton);
42 mainLayout->addWidget(hideHistoryButton);
43 mainLayout->addWidget(historyView);
44
45 historyWidget->setLayout(mainLayout);
46
47 QSplitter *mainSplitter = new QSplitter();
48 mainSplitter->addWidget(ui->centralWidget);
49 ui->centralWidget->setMinimumWidth(300);
50 mainSplitter->addWidget(historyWidget);
51 mainSplitter->setStretchFactor(1, 0);
52 mainSplitter->setStretchFactor(1, 0);
53 mainSplitter->setChildrenCollapsible(false);
54
55 QList<int> sizes;
56 sizes << 300 << 200; // ширина калькулятора и истории
57 mainSplitter->setSizes(sizes);
58
59 setCentralWidget(mainSplitter);
60 historyWidget->hide();
61

```

5) Добавлены слоты для действий показать, скрыть, очистить, сохранить для виджета истории. Также добавлен функционал подстановки выбранного значения из истории в displayPanel..

```

86
87 void MainWindow::onHideHistory() {
88     historyWidget->hide();
89     ui->centralWidget->setMinimumWidth(300);
90     resize(width() - historyWidget->width(), height());
91 }
92
93 void MainWindow::onShowHistory() {
94     if (historyWidget->isVisible()) return;
95     historyWidget->show();
96
97     historyWidget->setMinimumWidth(200);
98     resize(width() + historyWidget->width(), height());
99 }
100
101 void MainWindow::onClearHistory() {
102     historyList.clear();
103     historyView->clear();
104     ui->statusBar->showMessage("История очищена", 3000);
105 }
106
107 void MainWindow::onHistoryItemClicked (QListWidgetItem *item) {
108     QString value = item->text();
109     ui->displayPanel->setText(value);
110 }
111

```

```

142
143 void MainWindow::onSaveAction () {
144     QString currentValue = ui->displayPanel->text();
145     if (currentValue.isEmpty()) {
146         ui->statusBar->showMessage("Пустое значение не добавлено в историю.", 3000);
147         return;
148     }
149     historyList.append(currentValue);
150     ui->statusBar->showMessage("Значение " + currentValue + " добавлено в историю.", 3000);
151
152     // statusBarTimer->start(3000);
153     updateHistoryList();
154 }
155
156
157
160
161 void MainWindow::updateHistoryList () {
162     historyView->clear();
163     for (const QString& item : qAsConst(historyList)) {
164         historyView->addItem(item);
165     }
166 }
167

```

Все кнопки связаны с соответствующими слотами с помощью connect()

```

152
153
154 connect(hideHistoryButton, &QPushButton::clicked, this, &MainWindow::onHideHistory);
155 connect(clearHistoryButton, &QPushButton::clicked, this, &MainWindow::onClearHistory);
156 connect(historyView, &QListWidget::itemClicked, this, &MainWindow::onHistoryItemClicked);
157 connect(ui->Show_history, &QAction::triggered, this, &MainWindow::onShowHistory);
158 connect(ui->Hide_history, &QAction::triggered, this, &MainWindow::onHideHistory);
159 connect(ui->Save, &QAction::triggered, this, &MainWindow::onSaveAction);
160 connect(ui->About, &QAction::triggered, this, &MainWindow::onAbout);
161 //Доп задание
162 connect(ui->Export, &QAction::triggered, this, &MainWindow::onExport);
163 connect(ui->Import, &QAction::triggered, this, &MainWindow::onImport);
164
165
166 // statusBarTimer = new QTimer(this);
167 // statusBarTimer->start(3000);

```

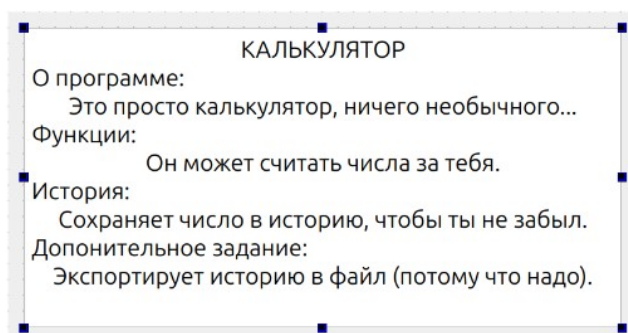
6) Для показа пункта О программе добавлен слот:

```

168 void MainWindow::onAbout () {
169     Form *info = new Form();
170     info->show();
171 }

```

7) Дизайн окна сделан в окне дизайнера



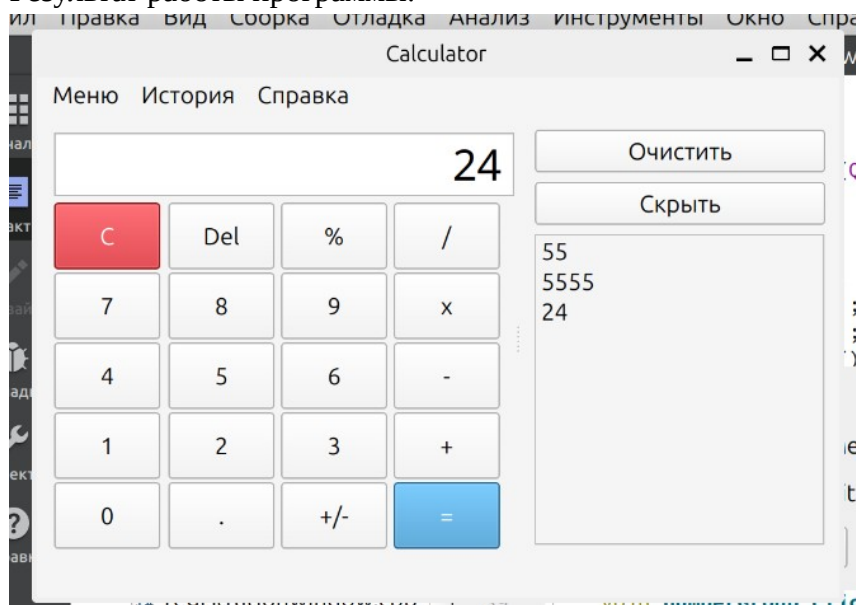
8)Дополнительное задание: сохранение и загрузка истории из файла

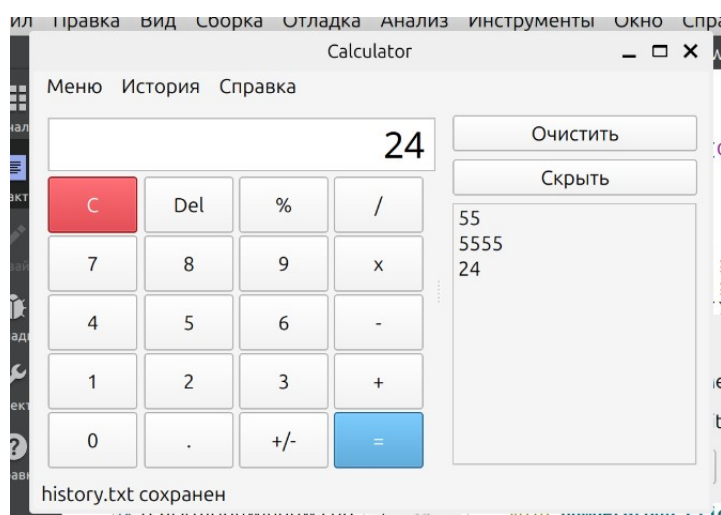
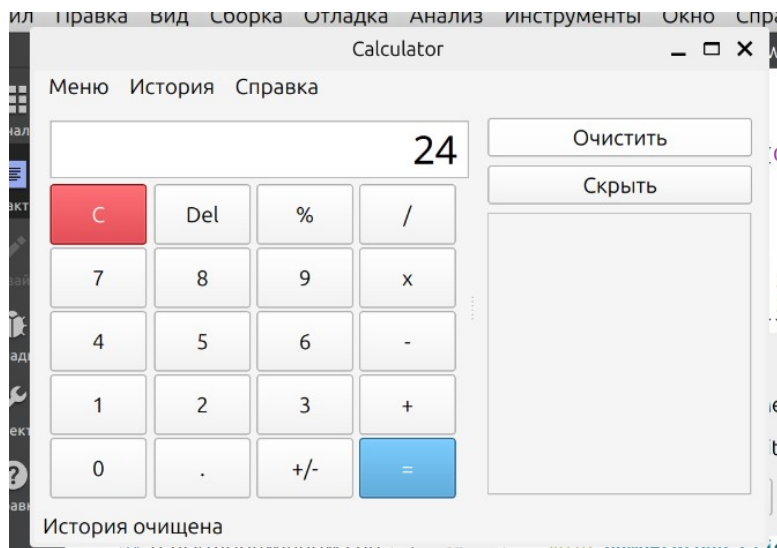
```
110 }
111
112 void MainWindow::onExport () {
113     QFile file("history.txt");
114     if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
115         QTextStream out(&file);
116         while (!historyList.isEmpty()) {
117             out << historyList.last() << "\n";
118             historyList.pop_back();
119         }
120         file.close();
121         ui->statusBar->showMessage("history.txt сохранен ");
122     }
123 }
124
125 void MainWindow::onImport () {
126     QFile file ("history.txt");
127     if(file.open(QIODevice::ReadOnly))
128     {
129         QTextStream stream(&file);
130         QString str;
131         while (!stream.atEnd())
132         {
133             str = stream.readLine();
134             historyList.append(str);
135         }
136         file.close();
137         ui->statusBar->showMessage("history.txt считан ");
138         updateHistoryList();
139     }
140 }
141 }
```

слоты экспорт и импорт открывают на запись и чтение

соответственно файл history.txt, после чего последовательно записывают(считывают) строки из списка в файл (и наоборот соответственно).

Результат работы программы:





Ответы на контрольные вопросы

Как работает механизм сигналов и слотов?

Сигнал отправляется объектом при наступлении события. Слот — это метод, который может быть вызван сигналом. Связь устанавливается через `connect()`.

Какие компоновщики (layout) используются в Qt и зачем они нужны?

Основные: `QVBoxLayout` (вертикальный), `QHBoxLayout` (горизонтальный), `QGridLayout` (сетка), `QFormLayout` (форма). Нужны для автоматического изменения размеров и позиций виджетов при изменении размеров окна.

Что такое `QSplitter` и в чём его отличие от других layout-компонентов?

`QSplitter` — это виджет, который делит пространство на части с подвижной границей (пользователь может перетаскивать разделитель). В отличие от обычных layout, он не просто управляет размерами, а даёт пользователю интерактивно менять пропорции.

Для чего используется `statusBar` в приложении? Примеры использования.

Для отображения временной или постоянной информации в нижней части главного окна. Примеры: "Файл сохранён", «Файл загружен»

Как работают горячие клавиши в Qt? Как их назначить для действий?

Через `QShortcut` или через `QAction::setShortcut()`. Для `QAction`: `action->setShortcut(QKeySequence::Open)` или `QKeySequence("Ctrl+S")`. При нажатии клавиши вызывается сигнал `activated()`.

Как реализовать реакцию на клик по элементу списка (`QListWidgetItem`)?

Подключиться к сигналам `QListWidget`: `itemClicked(QListWidgetItem*)`, `itemDoubleClicked()`, `currentItemChanged()`. В слоте обработать выбранный элемент.

Как сделать так, чтобы главное окно расширялось и сужалось при показе/скрытии боковой панели истории?

Использовать `setVisible()` для панели (виджета), а главное окно вызвать `adjustSize()` или переопределить `sizeHint()`. Либо использовать `QSplitter` с автоматическим обновлением.

Как добавить новую форму в проект?

В Qt Creator: ПКМ на проекте → "Add New..." → "Qt" → "Qt Designer Form Class" → выбрать тип (Main Window, Widget, Dialog) → задать имя. Будут созданы `.h`, `.cpp`, `.ui`.

Поясните как связана работа с `QAction` и `QMenu` в данном проекте.

В меню создаются ряд `QAction`, отвечающих за различные действия. Нажатие на пункт меню триггерит определенное действие. Например, сохранение элемента в истории.

Когда используются сигналы `triggered()` и `clicked()` в Qt?

`triggered()` — у `QAction` (вызывается при активации действия — клик, шорткат, программно).

`clicked()` — у кнопок (`QPushButton`, `QToolButton`, `QAbstractButton`). Для `QAction` `clicked()` не используется.